

We call the weighted sum of the rewards while following a policy from the starting state S_1 the *value* of that state, or a state value. We can denote this by the *value function* $V_\pi(s)$, which accepts an initial state and returns the expected total reward.

$$V_\pi(s) = \sum_{i=1}^{t} w_i R_i = w_1 R_1 + w_2 R_2 + \ldots + w_t R_t$$

The coefficients w_1, w_2, etc., are the weights we apply to the rewards before summing them. For example, we often want to weight more recent rewards greater than distant future rewards. This weighted sum is an expected value, a common statistic in many quantitative fields, and it's often concisely denoted $E[R|\pi,s]$, read as "the expected rewards given a policy π and a starting state s." Similarly, there is an *action-value function*, $Q_\pi(s,a)$, that accepts a state S and an action A and returns the value of taking that action given that state; in other words, $E[R|\pi,s,a]$. Some RL algorithms or implementations will use one or the other.

Importantly, if we base our algorithm on learning the state values (as opposed to action values), we must keep in mind that the value of a state depends completely on our policy, π. Using Blackjack as an example, if we're in the state of having a card total of 20, and we have two possible actions, hit or stay, the value of this state is only high if our policy says to stay when we have 20. If our policy said to hit when we have 20, we would probably bust and lose the game, so the value of that state would be low. In other words, the value of a state is equivalent to the value of the highest action taken in that state.